

Recruiter-Brain Candidate Ranker

Redrob — Intelligent Candidate Discovery & Ranking

Rank the top 100 of 100,000 candidates against one JD, best-fit first, each with an honest fact-grounded reason — on CPU, < 5 min, no network.

Thesis: the edge is *understanding the role*, not matching keywords.

The real problem isn't search — it's judgment under traps

- The JD is mostly **negative signal** (disqualifiers, anti-patterns). A keyword/embedding matcher ranks the people the JD explicitly rejects.
- The pool is **adversarial**: keyword stuffers, internally-impossible honeypots, consulting-only careers, plain-language strong candidates with no buzzwords.
- Hard rank-time limits: **≤5 min, 16 GB, CPU-only, no network**.
- Human multi-stage review (reproduce → manual → interview) is built to fail paste-and-pray submissions.

Measured: 8,545 candidates carry 3+ relevant skills, but only 755 have a corroborating title → **~10:1 keyword-stuffer noise**. "Has the skills" is not discriminative.

The signature move: a JD-understanding layer

One **structured, human-reviewed rubric** (`rubric.yaml`), extracted from the JD offline with an LLM then frozen, drives three things at once:

1. **Ranking** — what the role actually rewards
2. **Honeypot avoidance** — internal-consistency rules
3. **Reasoning** — every clause traces to a profile fact

The **negatives are the differentiator** — most teams encode only positives. Each disqualifier maps to a *detectable pattern*, not a vibe.

Anti-pattern	Detection signal
Keyword stuffer	relevant skills + unrelated title/career, AI skills dated < 12–18 mo
Consulting-only	every employer a services firm, no product company ever
Honeypot	role months > total career; "expert" in N skills with ~0 months used
CV/speech, no NLP/IR	skill set centered on vision/speech, no retrieval/NLP

Architecture: intelligence offline, rank-time fast & model-free

OFFLINE (network OK, no clock)

JD --LLM--> rubric.yaml (frozen)
candidates.jsonl --> features.parquet
42 raw signals + TF-IDF semantic_sim

RANK TIME (≤ 5 min, CPU, no net)

load rubric.yaml
score = weighted sum over features
sort, take top 100
reason from facts → submission.csv

- features.parquet holds **raw** signals; the scorer applies rubric **weights**. Tuning re-runs only the ~1s scorer, never the 90s extraction.
 - No model loaded at rank time → the 5-min budget is never at risk.
 - TF-IDF semantic layer: zero heavy deps, fully reproducible, drop-in for BGE later.
-

Scoring: transparent composition

```
final = base_fit (domain coherence + must-haves + semantic)
    × behavioral_availability      # responsive, active, open_to_work
    - disqualifier_penalties
    ; honeypot_hard → hard floor
```

- **Domain coherence (title ↔ career ↔ skills) is the #1 feature** — weighted far above raw skill tags (title-match 3.0 vs skill-tag 0.4). It's the highest lever on NDCG@10.
 - Disqualifiers are **soft penalties** (pull polished-fakes *out of* the top 10), not hard filters (which would nuke plain-language strong candidates).
 - Honeypots: **precision over recall** — two genuinely-impossible rules as a hard floor (~40 candidates), not broad rules that demote legit seniors.
-

Evaluation: we built our own (no leaderboard)

The JD explicitly asks for "designing evaluation frameworks for ranking systems." Doing it *is* demonstrating the skill being hired for.

- Hand-labeled candidates into relevance tiers 0–4; scored with NDCG@10/@50, MAP, P@10.
 - **Subset NDCG@10 = 0.9968** (NDCG@50 0.9968, MAP 0.9667) — **flat across every weight knob** → saturated and robust, not overfit. Config locked; no weight-bending on a small label set.
 - **Top-100 honeypot rate = 0%** (pool has 40) — re-runnable gate, `eval/check_traps.py`, exits non-zero if it ever crosses 10%.
 - Top-100: 100/100 domain-corroborated, YoE median 6.6 (in the 5–9 band).
-

Reasoning: specific, honest, varied — from real facts

Deterministic at rank time (no LLM, no network). Every clause traces to a profile column; skills named only from the candidate's own list; gaps acknowledged honestly.

Rank 1 — 7.2y Senior Machine Learning Engineer; the career text backs up the AI/retrieval signal, not just skill tags. Production embeddings/retrieval, vector-DB & hybrid search, ranking evaluation (NDCG/MRR/MAP)...

Rank 41 — 6.5y AI Research Engineer with title-and-summary evidence of real in-domain work. Vector-DB & hybrid search, shipping at scale, strong Python... **Gap:** no explicit embeddings-retrieval evidence.

100/100 reasons unique. Tone tracks rank; lower ranks name the top missing must-have.

Why it wins each stage

- **Stage 2 (score):** coherence + behavioral sharpen the top-10, where 50% of the composite lives.
- **Stage 3 (sandbox + honeypots):** precomputed artifacts → fast, reproducible; honeypots floored out of the top 100.
- **Stage 4 (reasoning + git):** reasons from real facts; genuine phased commit history.
- **Stage 5 (interview):** every number is explainable. We built a system, not a prompt.

Reproduce: `python rank.py --candidates ./candidates.jsonl --out ./submission.csv`